

# Optimal execution by differentiable programming

## Abstract

We show that optimal execution under nonlinear transient market impact can be represented as a transformer sequence model. The propagator becomes causal attention; the execution strategy’s cost becomes the forward pass through a neural network. This opens up the possibility of using a neural-network-optimized stack, such as PyTorch and the Adam optimizer on Nvidia GPUs, to compute optimal trading paths in the propagator model, as well as to train model parameters from realized executions of metaorders. For power-law impact and decay, our method matches or improves on the table of optimal values reported by Curato, Gatheral, and Lillo (2017), and on signed schedules reaches those values significantly faster than the sequential quadratic programming method. A single batched descent allows us to map signed-acquisition costs together with round-trip manipulation certificates across a plane parameterized by the impact and propagator exponents. A similar batched computation locates the slippage frontier at a manipulable exponent pair, where slippage renders the signed execution problem well posed.

**Keywords:** optimal execution; transient market impact; propagator model; price manipulation; attention mechanisms; differentiable programming

**JEL classification:** C61; G14; C45; C63; G11

## Highlights

- The propagator model is a transformer sequence model
- Adam optimizer matches or lowers published optima
- Batched Adam allows mapping execution cost landscape on NVIDIA GPUs

## 1. Introduction

A robust body of empirical work has established that the price response to a metaorder builds up during execution and partially decays afterwards, with a slow, approximately power-law relaxation (Lillo, Farmer, and Mantegna, 2003; Bouchaud et al., 2004; Almgren et al., 2005; Bouchaud et al., 2018). The propagator model, introduced in discrete time by Bouchaud et al. (2004) and formulated in continuous time with nonlinear instantaneous impact and general decay by Gatheral (2010), captures both features. In Gatheral’s form, the price  $S(t)$  evolves as

$$S(t) = S(0) + \int_0^t f(v(s)) G(t-s) ds + \int_0^t \sigma dW_s, \quad (1)$$

where  $W$  is a Brownian motion,  $v(s)$  is the signed rate of trading in shares per unit time,  $f$  is the instantaneous impact function, and  $G$  is a decay kernel encoding the transience of impact.

Meanwhile, the transformer architecture of Vaswani et al. (2017) has become the dominant model class in machine learning for sequential data. Transformer models apply the *attention mechanism*, in which the output at position  $t$  is a weighted aggregate of transformed inputs, called *values*, at positions  $s \leq t$ , with weights that are themselves functions of the data at  $t$  and  $s$ . Informally, position  $t$  *attends to* the positions before it. The propagator model (1) possesses a structure reminiscent of the attention mechanism: the impact component of the price at time  $t$  is a weighted aggregate over past trades; the weight attached to time  $s$  is  $G(t - s)$ ; the “value” extracted from time  $s$  is  $f(v(s))$ . The transient nature of impact, with later price moves “attending to” the impact of earlier trades, can be represented as an attention profile. Making this correspondence precise turns out to be productive. Our contributions are as follows.

- (i) Section 2 maps the propagator model onto the terminology of modern machine learning, specifically the class of models known as transformer sequence models. The propagator kernel corresponds to a state-dependent attention weight  $A_\theta(x_t, x_s, t - s)$ , where  $\theta$  denotes the full parameter vector. Each token  $x_t = (v_t, c_t)$  contains the trading rate  $v_t$  alongside market covariates  $c_t$ . The classical impact function  $f(v)$  generalizes to a map  $f_\theta(x_s)$  depending on the full token.
- (ii) Sections 3.1 and 3.2 form the computational core of the paper. It is shown that the expected cost of a trading schedule is the forward pass of a causal neural network. It suffices to work with a smooth approximation (26) in which automatic differentiation supplies exact gradients. Optimal execution becomes gradient descent in schedule space, using the same computational graph that fits the model to data.
- (iii) Section 3.3 describes batched Adam descent for this problem. This is not a fundamentally new algorithm, but it takes sophisticated techniques from the machine learning literature (Paszke et al., 2019; Kingma and Ba, 2015), adapts them, and applies them to finance in a new way. In this algorithm, schedules, restarts and model parameters occupy independent tensor dimensions, and hence a family of independent optimizations runs through gradient descent as one batched program in PyTorch with the Adam optimizer.
- (iv) Sections 3.4 to 3.6 exploit the powerful computational tools described in the earlier sections to improve upon the table of values published in Curato, Gatheral, and Lillo (2017). One batched descent of 4,032,000 schedules reaches every published optimum and lowers eleven of the twenty-one signed costs, in 45 seconds on one NVIDIA H100.
- (v) Sections 3.7 and 3.8 map the signed-acquisition cost surface, the slippage surface, and bound the zero-net-quantity manipulable region. Because the quadrature is exact on piecewise-constant schedules, every negative signed cost certifies that acquisition schedule and every negative zero-net-quantity cost certifies manipulation in the continuous-time model.
- (vi) Section 4 shows how batched Adam descent could be used to train the model on metaorder data, and Section 5 discusses open problems suggested by the generalized transformer architecture.

In related work, Leal, Laurière, and Lehalle (2022) parameterize a trading curve by a neural network and train it by stochastic gradient descent in an Almgren–Chriss setting. Section 3 instead optimizes the rate in each time bin directly and applies gradient-based optimization to nonlinear transient impact, where the cost to be minimized is the forward pass of the impact network itself and the results can be benchmarked against the brute-force optima of Curato, Gatheral, and Lillo (2017).

Closest in spirit to Section 3.7, Tsaknaki, Macrì, and Lillo (2026) show that a reinforcement-learning agent discovers price manipulation in an Almgren–Chriss model with nonlinear permanent impact; our searches operate in the transient propagator model with exact quadrature and map signed-acquisition costs together with round-trip manipulation certificates across the whole exponent plane. Neuman, Stockinger, and Zhang (2023) consider nonparametric estimation of the propagator from a dataset containing correlated price trajectories, trading signals and metaorders via reinforcement learning.

## 2. The transformer-propagator model

The propagator model of Bouchaud et al. (2004) and Gatheral (2010) is a causal attention head whose weights are unnormalized. Section 2.1 states the model in continuous time and fixes the exponent families used in every experiment below, Section 2.2 puts it on a grid with weights for which the discrete cost equals the continuous-time cost of a piecewise-constant path, and Section 2.3 gives the attention form together with a condition on the attention weight under which no round trip has negative cost.

### 2.1. The standard propagator model

Let  $S(t)$  be the mid-price of a single asset and  $v(t)$  the trader’s signed rate of execution at time  $t$ , in shares per unit time, positive for buying and negative for selling. The propagator model of Bouchaud et al. (2004) and Gatheral (2010) writes the price as the arrival price plus the accumulated impact of past trading plus noise, in the form (1) of the introduction,

$$S(t) = S(0) + \int_0^t f(v(s)) G(t-s) ds + \int_0^t \sigma dW_s.$$

The *instantaneous impact*  $f$  sends a trading rate to the immediate price pressure it exerts, and the *decay kernel*  $G$ , also called the propagator, gives how long that pressure lasts, a trade at time  $s$  still moving the price at  $t > s$  by the factor  $G(t-s)$ . We adopt the standing assumptions of Gatheral (2010).

Huberman and Stanzl (2004) show that no-dynamic-arbitrage requires that if market impact is permanent, then

$$f(-v) = -f(v) \text{ for all } v \in \mathbb{R}. \quad (2)$$

Gatheral (2010) reports consistency of (2) with his own empirical observation and thereafter takes it as a standing assumption. Thus we fix the following conventions throughout the paper.

**Definition 2.1.** An *instantaneous impact function* shall be a continuous, nondecreasing function  $f : (0, \infty) \rightarrow (0, \infty)$  such that  $\lim_{v \rightarrow 0^+} f(v) = 0$ . We extend it to  $\mathbb{R}$  by

$$f(0) = 0 \quad \text{and} \quad f(-v) = -f(v) \quad \text{for } v > 0.$$

Write the linear impact law as

$$f(v) = \lambda v, \quad \lambda > 0, \quad (3)$$

and call  $\lambda$  the *impact coefficient*. It is in units of currency per share squared, and it is the transient counterpart of the linear price-impact coefficient of Kyle (1985). In the linear model, execution data determine the response curve  $\tau \mapsto \lambda G(\tau)$ , and the normalization of  $G$  determines which point of that curve is reported as  $\lambda$ .

**Definition 2.2.** A *propagator* shall be a nonincreasing function  $G : (0, \infty) \rightarrow [0, \infty)$  that is integrable on  $(0, a]$  for every  $a > 0$ . Every propagator shall be implicitly extended to the domain  $\mathbb{R} \setminus \{0\}$  by using the even extension, i.e.

$$G(\tau) := G(-\tau) \quad \text{if } \tau < 0. \quad (4)$$

Empirically  $f$  is concave and  $G$  decays slowly, approximately as a power law (Lillo, Farmer, and Mantegna, 2003; Almgren et al., 2005; Bouchaud et al., 2018). That observation fixes the family used in every numerical experiment below, and in the results that describe them.

**Definition 2.3.** The *dual power-law model* with decay exponent  $\gamma \in (0, 1)$ , impact exponent  $\delta \in (0, 1]$  and impact coefficient  $\lambda > 0$  is the pair

$$f(v) = \lambda \operatorname{sgn}(v) |v|^\delta, \quad G(\tau) = \tau^{-\gamma}. \quad (5)$$

Both restrictions on the exponents in Definition 2.3 are needed for (5) to be admissible.

**Definition 2.4.** Let  $\gamma^*$  denote the solution in  $(0, 1)$  of  $2^{2-\gamma} = 3$ , that is

$$\gamma^* = 2 - \frac{\log 3}{\log 2} \approx 0.4150. \quad (6)$$

To show that manipulation is attainable whenever  $\gamma < \gamma^*$ , Gatheral (2010) takes an instantaneous impact that diverges as the trading rate approaches the depth of the order book. The impact of (5) stays finite, so attainability under it is left open there. Section 3.7 settles part of that question by exhibiting schedules.

Consider a horizon of  $T > 0$  units of clock time  $t_c$ , with both  $t_c$  and  $T$  measured in days. We normalize time once and for all, adopting the dimensionless time

$$t := t_c/T \in [0, 1] \quad (7)$$

as the time variable of the entire paper. A *trading-rate path*  $v : [0, 1] \rightarrow \mathbb{R}$  specifies at each  $t$  the instantaneous trading rate  $v(t)$ , in shares per unit of dimensionless time, so that its integral over a subinterval of  $[0, 1]$  is a number of shares; a clock-time rate of  $\nu(t_c)$  shares per day corresponds to  $v(t) = T \nu(tT)$ . In practice, some execution schedules are built for a horizon of a single trading day, with clock time quoted in days, and then the dimensionless time  $t$  coincides numerically with  $t_c$ .

**Definition 2.5.** Let  $v$  be a trading-rate path. Its *expected implementation shortfall* is

$$\mathcal{C}[v] := \mathbb{E} \int_0^1 v(t) (S(t) - S(0)) dt.$$

A *round trip* is a path satisfying  $\int_0^1 v(t) dt = 0$ . A round trip is *price manipulation* when  $\mathcal{C}[v] < 0$ .

## 2.2. The discrete propagator model

Split  $[0, 1]$  into  $N$  intervals of width  $\Delta = 1/N$  with grid points  $t_n = n\Delta$ ,  $n = 0, \dots, N$ . Let  $v_m$  be the constant rate on interval  $m$ , and let  $M_n$  denote the sampled price noise, a martingale. The discrete propagator model is usually written by sampling the kernel once per lag (Bouchaud et al., 2004), and we give it in that form to fix the impact state before replacing the weights. Define the endpoint-averaged kernel weights

$$G_k^{(\Delta)} := \frac{1}{\Delta} \int_{t_k}^{t_{k+1}} G(u) du, \quad 0 \leq k \leq N-1. \quad (8)$$

Evaluating the piecewise-constant path at interval endpoints gives the discrete propagator model

$$S_n = S_0 + \sum_{m \leq n} G_{n-m}^{(\Delta)} f(v_m) \Delta + M_n. \quad (9)$$

The impact terms group into a single quantity, called the *impact state*  $I_n$ ; see Webster (2023) for an expository account and references:

$$S_n = S_0 + I_n + M_n, \quad (10)$$

$$I_n := \sum_{m \leq n} G_{n-m}^{(\Delta)} f(v_m) \Delta. \quad (11)$$

The impact state  $I_n$  collects the price pressure outstanding at interval  $n$  from all trading in earlier intervals.

The propagator, the impact function, or both are typically specified as parametric functional forms, so in principle every one of them depends on the parameters of its family. It is usual to leave those parameters implicit, as in Gatheral (2010). Since we are discussing models which can train those parameters, we write  $\theta$  for the full parameter vector of the model and attach it as a subscript to whatever it determines. Note that Section 3.2 holds  $\theta$  fixed and optimizes over schedules while Section 4 does the reverse.

A particularly convenient class of propagators is the multi-exponential class. Fix  $K$  modes with weights  $\omega_k > 0$  and rates  $\rho_k > 0$ , and take

$$G_\theta(\tau) = \sum_{k=1}^K \omega_k e^{-\rho_k \tau}. \quad (12)$$

In this case, one block of the parameter vector  $\theta$  contains the parameters  $\{\omega_k, \rho_k : k = 1, \dots, K\}$ . Every mode is a one-state linear recurrence, so the impact state splits into  $K$  scalar states,  $I^{(k)}, k = 1, \dots, K$  with

$$I_n = \sum_k \omega_k I_n^{(k)} \quad (13)$$

$$I_n^{(k)} = e^{-\rho_k \Delta} I_{n-1}^{(k)} + \frac{1 - e^{-\rho_k \Delta}}{\rho_k \Delta} f(v_n) \Delta, \quad I_0^{(k)} = 0. \quad (14)$$

When we do cost-minimization in discrete time, the functional that is minimized is the exact continuous-time cost  $\mathcal{C}[v]$  computed on a piecewise-constant path. This has the advantage that a negative-cost schedule found by discrete-time optimization is also a particular example of a negative-cost continuous-time path.

Substituting (1) into Definition 2.5 and using that the noise is a martingale, gives a double integral:

$$\mathcal{C}[v] = \int_0^1 v(t) \int_0^t f(v(s)) G(t-s) ds dt. \quad (15)$$

Both  $t$  and  $s$  run over the same grid, so on a piecewise-constant schedule the double integral collapses to a finite sum whose coefficients depend on the model but not on the schedule. Write  $B_n = [(n-1)\Delta, n\Delta]$  for the  $n$ -th bin and put

$$W_k := \int_{B_n} \int_{B_m \cap [0,t]} G(t-s) ds dt, \quad k = n - m \geq 0. \quad (16)$$

The integrand depends on  $t$  and  $s$  only through  $t-s$  and the bins are equally spaced, so the value depends on  $n$  and  $m$  only through  $k$ .

**Lemma 2.6.** *Let  $v$  be constant and equal to  $v_m$  on each bin  $B_m$ . Then*

$$\mathcal{C}[v] = \sum_{n=1}^N \sum_{m \leq n} v_n W_{n-m} f(v_m). \quad (17)$$

*Proof.* Split the outer integral of (15) over the bins and the inner one over the bins that precede  $t$ . On  $B_n$  the rate is the constant  $v_n$ , and on  $B_m$  the value  $f(v(s))$  is the constant  $f(v_m)$ , so both factor out of the integrals:

$$\mathcal{C}[v] = \sum_{n=1}^N \int_{B_n} v_n \sum_{m \leq n} \int_{B_m \cap [0,t]} f(v_m) G(t-s) ds dt = \sum_{n=1}^N \sum_{m \leq n} v_n f(v_m) \int_{B_n} \int_{B_m \cap [0,t]} G(t-s) ds dt,$$

and the remaining integral is  $W_{n-m}$  by (16). For  $m < n$  the whole of  $B_m$  precedes every  $t \in B_n$ , so the restriction  $B_m \cap [0, t]$  is  $B_m$ ; at  $m = n$  it retains the causal half of the square.  $\square$

**Definition 2.7.** Let  $\theta$  be fixed and let  $v = (v_1, \dots, v_N) \in \mathbb{R}^N$  collect the rates on the grid of Section 2.1. The *discrete expected implementation shortfall* of  $v$  is

$$C_\theta[v] := \sum_{n=1}^N \sum_{m \leq n} v_n W_{n-m} f(v_m). \quad (18)$$

Script letters denote quantities defined on the continuum and roman letters their counterparts on the grid. By Lemma 2.6,  $C_\theta[v] = \mathcal{C}[v]$  whenever  $v$  is piecewise constant on the grid.

Writing  $t$  and  $s$  as distances into their bins, the difference  $t-s$  has the triangular density  $\Delta - |w|$  on  $[-\Delta, \Delta]$ , so

$$W_0 = \int_0^\Delta (\Delta - w) G(w) dw, \quad (19)$$

$$W_k = \int_{-\Delta}^\Delta (\Delta - |w|) G(k\Delta + w) dw, \quad k \geq 1.$$

The lag-zero weight has half the mass of the others, because only  $s < t$  contributes within a single bin. Closed forms follow in Section 3.1. The bin-integrated weights (16) differ from the sampled weights (8). For example at lag zero, the latter assigns  $\Delta^{2-\gamma}/(1-\gamma)$  while the former is  $\Delta^{2-\gamma}/((1-\gamma)(2-\gamma))$ , which is larger by the factor  $2-\gamma$ . Every cost reported in this paper comes from (18).

### 2.3. Impact is an unnormalized kernel-attention head

We associate to the  $m$ -th interval the following vector,

$$x_m = (v_m, c_m) \in \mathbb{R}^d,$$

containing the trading rate  $v_m$  in the first coordinate, and covariates  $c_m \in \mathbb{R}^{d-1}$  in the remaining coordinates. The exact selection of covariates that are used is a modeling choice. Each trader may select covariates that could help forecast market impact. Some examples include volatility, spread, depth, imbalance, volume, signed flow, and time of day.

In sequence-based machine learning models  $x_m$  is called a *token*. In the classic propagator model,  $f$  would depend on the token  $x_m$  only through its first component  $v_m$ , while  $G$  would depend on a token pair only through their time separation. Embedding this class of models within the broader class of transformer models provides a viable path to state-dependent generalizations. In the present work, we focus on machine-learning-native numerical methods for the classical model and their performance relative to classical optimization methods.

**Definition 2.8.** The *transformer-propagator model* is defined to be a direct generalization of the discrete propagator model (10)–(11) where (10) is not changed, and the impact state (11) is replaced with the more general form

$$I_n = \sum_{m \leq n} A_\theta(x_n, x_m, (n - m)\Delta) f_\theta(x_m) \Delta. \quad (20)$$

Here,  $A_\theta(x_n, x_m, (n - m)\Delta)$  is a generalized propagator. Like the classical propagator, it depends on the time separation  $(n - m)\Delta$  but it is more general because it also allows a dependence on the tokens  $x_n, x_m$ . From (20), the classical form (11) is recovered by assuming that  $f_\theta$  depends on  $x_m$  only through  $v_m$ , and setting  $A_\theta(x_t, x_s, \tau) = G_\theta(\tau)$ .

The form (10) with the generalized impact state (20) can be directly compared to the class of neural network architectures known as transformer models. The impact function  $f_\theta(x_m)$  is the *value* at position  $m$ , and it has been promoted to be a function of the whole market state, while in the standard propagator model, the impact function is taken to be a function of the trading rate alone. Represented this way,  $I_n$  is an example of a machine-learning construct known as an *attention head*.

The generalized propagator  $A_\theta(x_n, x_m, (n - m)\Delta)$  is known in machine learning (Bahdanau, Cho, and Bengio, 2015) as the *attention weight*. Rather than allowing arbitrarily complex forms for the attention weight, following Tsai et al. (2019) we suggest that the dependence on  $\tau$  factors out, and the dependence on tokens  $x_t, x_s$  comes from an inner product, in the following way:

$$A_\theta(x_t, x_s, \tau) = \mathcal{K}_\theta(x_t, x_s) G_\theta(\tau), \quad \text{where:} \quad (21)$$

$$\mathcal{K}_\theta(x, y) := \langle \Phi_\theta(x), \Psi_\theta(y) \rangle$$

where  $\Phi_\theta, \Psi_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^p$  are called the *query map* and the *key map* respectively, and where  $G_\theta$  is a propagator in the sense of Definition 2.2. Weights of this form multiply the values and enter the sum (20) directly. Thus (20) is an unnormalized kernel-attention head, linear in its values. It shares the query-key feature-map factorization used by the linear-attention models of Katharopoulos et al. (2020). Throughout we take  $\Psi_\theta = \Phi_\theta$ , so that  $\mathcal{K}_\theta$  becomes symmetric and positive semidefinite. Figure 1 illustrates the proposed network structure, taking  $\Psi_\theta = \Phi_\theta$  and the diagonal matrix  $D$  built from a multi-exponential kernel (12).

Write  $\mathcal{C}_\theta[v]$  for the shortfall of Definition 2.5 when the price has the generalized impact state (20).

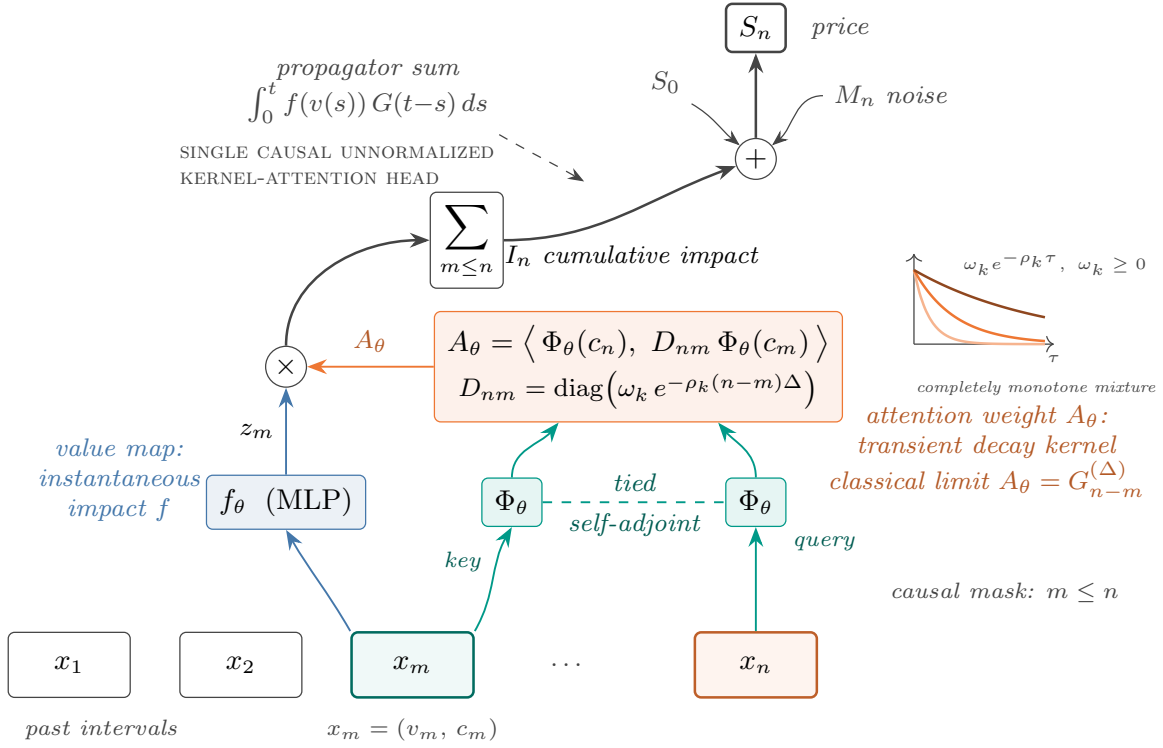


Figure 1: The transformer-propagator model (20) with multi-exponential kernel (12) as a causal unnormalized kernel-attention head. Blue: nonlinear value map. Teal: tied query and key map. Orange: state-dependent transient kernel.

**Proposition 2.9.** *Let  $A_\theta$  be of the form (21) with  $\theta$  fixed, and  $f(v) = \lambda v$  with  $\lambda > 0$ . Let  $G_\theta$  be completely monotone and let  $\mathcal{K}_\theta$  be a continuous positive semidefinite kernel. Every piecewise-continuous trading-rate and token path whose cost is absolutely convergent satisfies  $\mathcal{C}_\theta[v] \geq 0$ . In particular, every such round trip has nonnegative expected cost.*

*Proof.* Symmetrizing the double integral, the expected cost of a schedule is

$$\mathcal{C}_\theta[v] = \frac{\lambda}{2} \int_0^1 \int_0^1 \mathcal{K}_\theta(x_s, x_t) G(t-s) v(s) v(t) ds dt.$$

By the Bernstein representation and complete monotonicity (Widder, 1941; Schilling, Song, and Vondraček, 2012),  $(s, t) \mapsto G(t-s)$  is a positive semidefinite kernel. A product of positive semidefinite kernels is again one, by the Schur product theorem, and the double integral is therefore nonnegative for every  $v$ .  $\square$

*Remark 2.10.* Positive semidefiniteness of  $\mathcal{K}_\theta$  is the main hypothesis of Proposition 2.9, and for that we need  $\Psi_\theta = \Phi_\theta$ , in which case we call  $\Psi_\theta, \Phi_\theta$  a “tied pair.” Over  $n$  market states a tied pair produces the Gram matrices of  $n$  vectors in  $\mathbb{R}^p$ , which for  $p \geq n$  are exactly the positive semidefinite  $n \times n$  matrices. An untied pair produces every  $n \times n$  matrix, since  $\Phi_\theta$  may run through the standard basis of  $\mathbb{R}^n$  while  $\Psi_\theta$  reads off the columns of whatever matrix is wanted. A profitable round trip therefore needs only a symmetric  $\mathcal{K}_\theta$  along which the product kernel

$\mathcal{K}_\theta(x_s, x_t) G(t - s)$  turns negative for some path of zero net quantity. Two states whose inner product is smaller within each state than across the pair already suffice: a trader who buys in the first and sells in the second pays the small within-state weight on each leg and collects the large cross weight on the reversal, and the collection exceeds the payment once the within-state weight is small enough.

To summarize in machine-learning terms, the model is a single causal kernel-attention head whose attention pattern resembles a decoder-only Transformer.

### 3. Optimal execution by differentiable programming

Reverse-mode differentiation of (18) solves two problems. In the first,  $f$  and  $G$  are given and we want the cheapest schedule and the cost it attains. With  $\theta$  fixed and a target quantity  $X$ , this is

$$\min_v C_\theta[v] \quad \text{subject to} \quad \Delta \sum_{n=1}^N v_n = X, \quad (22)$$

on the grid of Section 2.1, with  $C_\theta$  the model cost. In the second,  $f$  and  $G$  are unknown and we infer them from execution data by minimizing a loss over  $\theta$ , subject to the shape restrictions of Definitions 2.1 and 2.2.

Solving for a schedule varies  $v$  at fixed  $\theta$ ; learning the model varies  $\theta$  with a large number of training samples, where each sample contains a particular fixed schedule  $v$ , corresponding to a realized execution profile. Both computations minimize a scalar produced by the same forward pass, and the tooling built to train neural networks handles both.

The results reported in this section split naturally into CPU calculations, which are generally the classical methods of Curato, Gatheral, and Lillo (2017), and calculations which naturally benefit from the GPU, such as batched Adam descent. The CPU methods were run on an Apple M2 with four performance cores, four efficiency cores and 24 GB of unified memory. All batched Adam calculations were run on one NVIDIA H100 PCIe with 80 GB, exclusive to the run. All calculations in this paper use Python 3.14, PyTorch 2.13.0, SciPy 1.18.0 and NumPy 2.5.1, in double precision throughout, with TF32 disabled on the graphics processor.

#### 3.1. The trading cost is a forward pass

Holding each token constant within its bin, the weight (16) generalizes to the causal head as

$$W_{nm}^\theta := \int_{B_n} \int_{B_m \cap [0, t]} A_\theta(x_n, x_m, t - s) ds dt, \quad m \leq n,$$

which reduces to  $W_{n-m}$  when the attention weight depends on the time separation alone. Writing  $\bar{I}_n = \Delta^{-1} \sum_{m \leq n} W_{nm}^\theta f_\theta(x_m)$  for the bin-average impact, Definition 2.7 becomes

$$C_\theta[v] = \sum_{n=1}^N v_n \sum_{m \leq n} W_{nm}^\theta f_\theta(x_m) = \Delta \sum_{n=1}^N v_n \bar{I}_n. \quad (23)$$

Computationally this is three operations: the value map applied to each token, the exact bin-integrated kernel matrix acting on the resulting sequence, and an inner product with the schedule.

Let  $J_G$  be a second antiderivative of the decay kernel. Integrating (16) gives the lag weights

$$\begin{aligned} W_0 &= J_G(\Delta) - J_G(0) - \Delta J'_G(0), \\ W_k &= J_G((k+1)\Delta) - 2J_G(k\Delta) + J_G((k-1)\Delta), \quad k \geq 1. \end{aligned} \quad (24)$$

Let  $\mathbf{W}$  be the lower-triangular Toeplitz matrix with  $\mathbf{W}_{n,m} = W_{n-m}$  for  $m \leq n$  and  $\mathbf{W}_{n,m} = 0$  for  $m > n$ . Then

$$C[v] = \sum_{n=1}^N \sum_{m \leq n} v_n f(v_m) W_{n-m} = v^\top \mathbf{W} f(v), \quad (25)$$

which is Lemma 2.6 written as a matrix product.

For the kernel of (5), define  $c_2 = 1/((1-\gamma)(2-\gamma))$ . Then

$$\begin{aligned} J_G(\tau) &= c_2 \tau^{2-\gamma}, \\ W_0 &= c_2 \Delta^{2-\gamma}, \\ W_k &= c_2 \Delta^{2-\gamma} ((k+1)^{2-\gamma} - 2k^{2-\gamma} + (k-1)^{2-\gamma}), \quad k \geq 1. \end{aligned}$$

For the shifted kernel  $G(\tau) = (\ell + \tau)^{-\gamma}$ , the same formulas use  $J_G(\tau) = c_2(\ell + \tau)^{2-\gamma}$  and  $J'_G(0) = \ell^{1-\gamma}/(1-\gamma)$ .

The impact function of (5) has an unbounded derivative at zero when  $\delta < 1$ . Gradient generation therefore uses the smooth odd surrogate

$$f_\varepsilon(v) = \lambda v (v^2 + \varepsilon)^{(\delta-1)/2}. \quad (26)$$

We take  $\varepsilon = 10^{-8}$  for Figure 3 and  $\varepsilon = 10^{-10}$  for the benchmark. Every reported cost evaluates the exact objective with the unsmoothed  $f$  at the schedule returned by the optimizer.

The cost is evaluated as a directed graph whose nodes are the elementary operations used to build  $C_\theta[v]$ . The forward pass computes each node and stores the intermediate quantities needed by its local derivative. In the machine learning literature, the *adjoint of a node*  $z$  means  $\partial C_\theta / \partial z$ . Reverse-mode automatic differentiation starts at the scalar output with adjoint 1, and traverses the graph backward. At each operation it multiplies the incoming adjoint by the operation's local derivative and adds the resulting contributions to the adjoints of its inputs. This is the chain rule applied one operation at a time. When the traversal reaches the schedule leaves, their adjoints are the  $N$  components of  $\nabla_v C_\theta[v]$ .

Every stored operation is processed a fixed number of times, so the work of the reverse sweep is proportional to the work of the forward pass, while its memory cost is the storage or recomputation of the forward intermediates (Griewank and Walther, 2008). Finite differences should not be used here, because they would perturb each trading rate separately and reevaluate the whole cost, requiring  $N+1$  forward passes for an  $N$ -bin schedule. For the smooth surrogate (26), one backward traversal therefore returns the exact gradient up to floating-point arithmetic.

Figure 2 illustrates the traversal for (25). Write  $y = f_\varepsilon(v)$ ,  $b = \mathbf{W}y$  and  $r = v \odot b$ , where  $\odot$  denotes componentwise multiplication. For any node  $z$ , write  $\bar{z} = \partial C / \partial z$  for its adjoint. The forward row computes  $C = \mathbf{1}^\top r$ . The reverse row applies the local derivative at each arrow. The two arrows entering  $\bar{v}$  give the contributions from the readout and the impact path, and their sum is  $\nabla_v C$ . These are the two terms in (27) below.

PyTorch (Paszke et al., 2019) performs this reverse traversal and adjoint accumulation automatically. A tensor flagged with `requires_grad` causes every tensor computed from it to retain the operation that produced it, so evaluating (23) also builds a graph of that evaluation, and calling `backward` on the resulting scalar walks the graph in reverse and deposits a derivative at

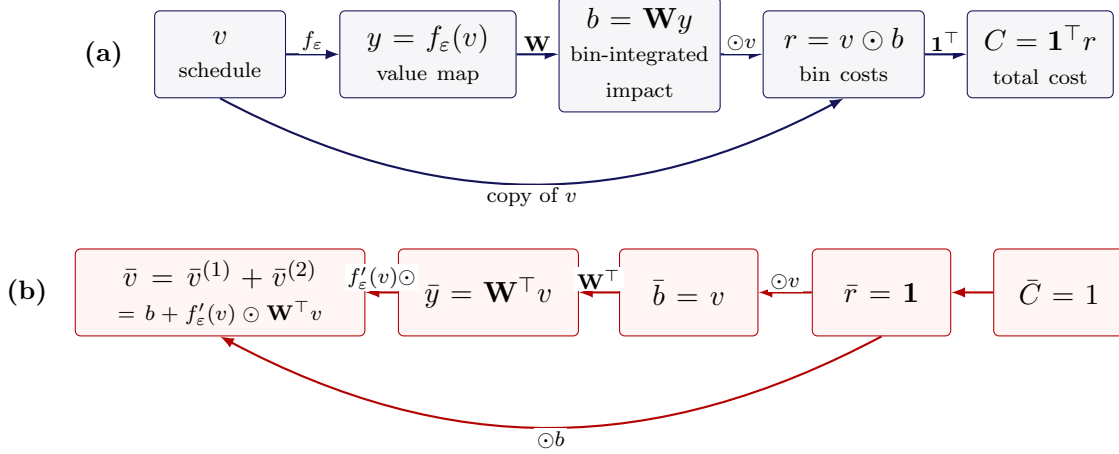


Figure 2: Reverse-mode differentiation of the discrete cost. (a) Forward evaluation. (b) Reverse propagation and accumulation of adjoints.

each flagged leaf. Which quantities get differentiated is decided by which tensors were flagged. Flag the schedule and the backward traversal returns  $\nabla_v C_\theta[v]$ ; flag the kernel and impact parameters instead and the same call returns the gradient of a fitting loss with respect to them. One implementation therefore serves both problems above. We use PyTorch and Adam (Kingma and Ba, 2015) throughout.

The gradient can also be written out. In the position-only case the chain rule gives

$$\frac{\partial C}{\partial v_i} = \sum_{m \leq i} W_{i-m} f(v_m) + f'(v_i) \sum_{n \geq i} v_n W_{n-i}. \quad (27)$$

Its constancy in  $i$  on the shares-constraint hyperplane is the discretized first-order condition of Curato, Gatheral, and Lillo (2017). Automatic differentiation evaluates (27) to machine precision and applies unchanged when the attention weight becomes state dependent.

The forward pass also admits an exact FFT formula. Set  $\mathbf{y} = (f(v_1), \dots, f(v_N))$  and  $\mathbf{w} = (W_0, \dots, W_{N-1})$ , with zeros appended so that  $\mathbf{y}, \mathbf{w} \in \mathbb{R}^{2N}$ . Let  $\mathcal{F}_{2N}$  denote the length- $2N$  discrete Fourier transform, let  $P_N$  retain the first  $N$  components, and define

$$b = P_N \mathcal{F}_{2N}^{-1} (\mathcal{F}_{2N}(\mathbf{w}) \odot \mathcal{F}_{2N}(\mathbf{y})). \quad (28)$$

The padding makes the circular convolution in (28) equal the linear convolution in its first  $N$  components, so  $b_n = \sum_{m \leq n} W_{n-m} f(v_m)$  and  $C[v] = v^\top b$ . The FFT evaluation costs  $O(N \log N)$ , compared with  $O(N^2)$  for multiplication by  $\mathbf{W}$ , and the two evaluations agree to relative error  $10^{-15}$  in double precision. The finest grids in Section 3.7 use (28).

### 3.2. Solving for a schedule

The simplest schedule-optimization algorithm seems to be to treat each bin separately, optimizing one unconstrained logit per time bin,  $w = (w_1, \dots, w_N)$ . We call this the *free-grid representation*, and is used in all our numerical examples. Write  $\bar{w} = N^{-1} \sum_n w_n$ .

**Definition 3.1.** Let  $F \subseteq \mathbb{R}^N$  be a set of admissible schedules. A *feasible-set map* for  $F$  is a differentiable map  $\Pi : \mathbb{R}^N \rightarrow \mathbb{R}^N$  whose image lies in  $F$  and is dense in  $F$ .

Every value of the free variable  $w$  produces an admissible schedule  $\Pi(w)$ , so descent on  $w$  is unconstrained and every iterate is a legitimate schedule. Since the image of  $\Pi$  is dense, we are guaranteed that the optimum can be found by optimizing over  $w \in \mathbb{R}^N$ . The two maps used in the experiments are the following.

**Example 3.2** (Signed acquisition). Let  $F = \{v \in \mathbb{R}^N : \Delta \sum_n v_n = X\}$  and

$$\Pi(w) = w - \bar{w} \mathbf{1} + X \mathbf{1}.$$

Rates of either sign are allowed. The image is the whole of  $F$ , and  $\Pi$  restricted to  $F$  is the identity, so  $\Pi$  is a projection onto  $F$ . Setting  $X = 0$  gives the round-trip constraint.

**Example 3.3** (Buy-only). Let  $F = \{v \in \mathbb{R}^N : v_n \geq 0, \Delta \sum_n v_n = X\}$ , a scaled simplex, and

$$\Pi_n(w) = NX \operatorname{softmax}_n(w) = NX \frac{e^{w_n}}{\sum_m e^{w_m}}.$$

The image is the set of schedules with every  $v_n > 0$ , which is the relative interior of  $F$  and is dense in it. A schedule on the boundary is therefore approached but never attained.

### 3.3. Batched Adam descent

The searches in this paper consist of many independent minimizations of the same functional form. They differ in where they start, in the settings given to the optimizer, and in the model parameters, but each optimizes its own objective and none of them communicate. Arranged as loops over those differences, they would run one after another, which is not required and is inefficient on modern hardware. In this section we describe a better way to organize such calculations that is particularly well suited for running on deep-learning hardware, such as the datacenter-class NVIDIA H100 GPU we used for this paper.

Index the independent problems by  $i$ , and let  $U_i \in \mathbb{R}^N$  hold the free variables of problem  $i$ , so that  $\Pi(U_i)$  is its schedule under the relevant feasible-set map of Definition 3.1 and  $C_{\theta_i}$  its cost. Collect the  $U_i$  into one array  $U$  and set

$$L(U) = \sum_i C_{\theta_i}[\Pi(U_i)]. \quad (29)$$

Each summand involves a single block of  $U$ , so

$$\frac{\partial L}{\partial U_i} = \frac{\partial}{\partial U_i} C_{\theta_i}[\Pi(U_i)],$$

the gradient of problem  $i$  alone. One reverse sweep over (29) therefore returns every gradient at once, and one optimizer step advances every problem exactly as it would have advanced by itself. Adam forms its update for a coordinate from running moments of that same coordinate, so a setting that varies between problems, the step size among them, travels with its block instead of forcing a separate run.

Taken one problem at a time, a schedule of  $N$  bins is a few hundred numbers, and a graphics processor spends longer receiving each instruction than executing it. Taken together, the same work becomes a handful of large array operations and the device runs at its bandwidth. Neural-network hardware and software are built around that property, and it works well here because Section 3.1 has already put the cost in the form they expect.

Any axis along which the problems are independent can be stacked this way. Restarts from different initial guesses, the optimizer settings swept alongside them, and whole grids of model parameters each with their own kernel all serve equally well, and they compose, so a grid of parameters crossed with a pool of restarts is one array rather than two nested loops. Section 3.6 optimizes  $21 \times 192,000$  schedules together on this principle and Section 3.7 does the same across a plane of exponents.

The listing below describes *batched Adam descent*. A *cell* is defined to be one particular setting of the model parameters; for example, an exponent pair  $(\gamma, \delta)$  in Section 3.6 and Section 3.7, or a friction pair  $(\eta_1, \eta_2)$  in Section 3.8. One array of logits has an axis for the parameter cell, an axis for the restart and an axis for the time bin, with a separate block for each step size, so one call to `backward()` produces every gradient and one optimizer step advances every problem. The arithmetic is the same as running the problems one at a time, but it is issued as a few large array operations instead of millions of small ones, which is the form a graphics processor executes at its memory bandwidth.

```
signed    = lambda u, X: u - u.mean(-1, keepdim=True) + X    # Example 3.2
buy_only  = lambda u, X: N * X * softmax(u, dim=-1)         # Example 3.3

# one block per step size, each of shape (cells, restarts, bins)
blocks = [randn(J, R, N, requires_grad=True) for _ in rates]
opt     = Adam([{"params": [b], "lr": r} for b, r in zip(blocks, rates)])

for _ in range(steps):
    opt.zero_grad(set_to_none=True)
    for b in blocks:
        # W is (cells, bins, bins)
        smooth_cost(signed(b, X), W).sum().backward()
    opt.step()
```

Every entry of a block enters the sum through its own summand, so the gradient of the sum is the stack of the *JR* individual gradients and one descent advances all optimizations in parallel. Creating the blocks with `requires_grad=True` makes every tensor built from them retain the operation that produced it, and `backward()` is then the reverse sweep of Section 3.1.

### 3.4. Search settings

This section gives the restart design and the search budgets in Table 1. The two classical methods run at the thousand-start design of Curato, Gatheral, and Lillo (2017).

Every restart begins at a fresh draw of the logits, with independent standard normal entries scaled by a common factor  $\beta$ , and the feasible-set map of Definition 3.1 sends that draw to a schedule. Under Example 3.2 the result is constant-rate trading displaced by a mean-zero Gaussian perturbation of standard deviation  $\beta$ ; under Example 3.3  $\beta$  is the standard deviation of the softmax logits, and a large value starts from a schedule concentrated in few bins. The searches spread a cell's restart budget over the product of two sets, the values of  $\beta$  and the Adam step sizes, since the step size that serves one cell best serves another poorly. The benchmark of Section 3.6 and the exponent plane of Section 3.7 hold each step size fixed for the whole descent. The slippage plane of Section 3.8 anneals it along a cosine schedule to one hundredth of its initial value, which lets it reach the same accuracy from fewer restarts per cell.

|                                         |                                             |
|-----------------------------------------|---------------------------------------------|
| <b>Batched descent of Section 3.6</b>   |                                             |
| starts per cell, signed                 | 192,000                                     |
| starts per cell, monotone               | 128,000                                     |
| steps                                   | 8,000                                       |
| step sizes                              | 0.01, 0.03, 0.08, 0.20                      |
| $\beta$ , signed                        | 0.05, 0.20, 1, 4, 16, 64                    |
| $\beta$ , monotone                      | 0.10, 0.50, 2, 8                            |
| gradient smoothing $\varepsilon$        | $10^{-10}$                                  |
| kernel contraction                      | dense; FFT above $N \approx 10^3$           |
| <b>Sequential quadratic programming</b> |                                             |
| implementation                          | SciPy <code>minimize</code> , method SLSQP  |
| starts, iteration cap, tolerance        | 1,000; 1,000; $10^{-10}$                    |
| <b>Generating-set direct search</b>     |                                             |
| starts, initial mesh                    | 1,000; 1                                    |
| expansion, contraction                  | 2; 1/2                                      |
| mesh and function tolerance             | $10^{-6}$ ; $10^{-12}$                      |
| poll                                    | opportunistic                               |
| <b>Exponent plane of Figure 6</b>       |                                             |
| coarse sweep                            | 4 restarts per cell, 3,000 steps            |
| refinement bands                        | 12 and 24 restarts, $\delta$ spacing 0.0025 |
| <b>Common</b>                           |                                             |
| starting points                         | Dirichlet, unit concentration               |

Table 1: Search budgets at  $X = 0.1$  and  $N = 100$ .

### 3.5. Signed schedules and manipulation tests

As a first test of the Adam optimizer, we use it to solve the classic optimal-execution problem. In other words, this is just classic trading-rate schedule optimization in the spirit of Curato, Gatheral, and Lillo (2017), but with a differentiable objective and all computations done in PyTorch with the Adam optimizer. We test the system in three regimes: linear impact, monotone schedules subject to a buy-only constraint, and a signed acquisition example that exhibits transaction-triggered price manipulation. Reassuringly, all of our results are as expected, validating the new optimizer.

At  $(\gamma, \delta) = (0.45, 0.60)$ , the exponents satisfy  $\gamma + \delta = 1.05$  and  $\gamma > \gamma^*$ . The signed acquisition search finds a schedule with expected cost  $-0.198$ : the trader acquires  $X$  and is paid to do so. Its gross turnover is 5.11 against a net of 1, and the schedule alternates buy bursts with partial unwinds, panel (c) of Figure 3. This bears a strong resemblance to the unconstrained optimum of Curato, Gatheral, and Lillo (2017): bursts of intense, short-lived buying separated by long stretches of slow selling, found there by thousand-start SQP and here by first-order descent. The deviation of this schedule from TWAP is a round trip with positive cost, so the negative cost lives in the cross terms between the round trip and the base load. This is transaction-triggered price manipulation in the sense of Alfonsi, Schied, and Slynko (2012): intermediate sells lower the expected cost of a buy program below that of every monotone schedule, and here below zero. Direct round-trip searches from twelve initializations have best cost  $1.8 \times 10^{-5}$ . At this

exponent pair, the search therefore certifies transaction-triggered manipulation and returns a round-trip cost close to zero.

For the shifted kernel  $G(\tau) = (0.01 + \tau)^{-0.45}$ , bounded at zero, the round-trip search exhibits a round trip of cost  $-71.6$  at gross turnover 192, and since the round-trip cost is homogeneous of degree  $1 + \delta$  in the schedule, any negative value scales to  $-\infty$ .

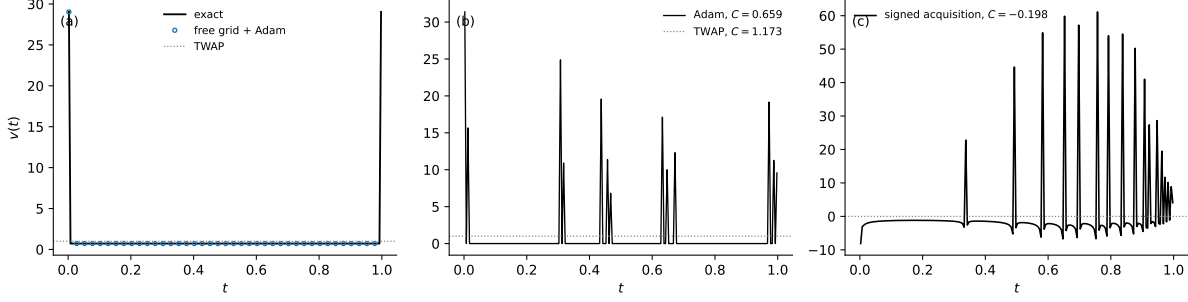


Figure 3: Schedules found by Adam. (a)  $f(v) = v$  and  $G(\tau) = e^{-5\tau}$ . (b) Buy-only schedule with  $f(v) = \text{sgn}(v)|v|^{0.6}$  and  $G(\tau) = \tau^{-0.45}$ . (c) Signed acquisition under the model in (b).  $N = 200$  and  $X = \lambda = 1$ ; costs appear in the panels.

### 3.6. Benchmark against brute-force global optimization

This section compares the batched descent of Section 3.3 against the two brute-force searches of Curato, Gatheral, and Lillo (2017) at the twenty-one parameter cells of their Table 2, on the cost attained and on the computation time.

Curato, Gatheral, and Lillo (2017) attacked the nonlinear problem by brute force: sequential quadratic programming (SQP) over signed schedules and a derivative-free generating set search, GSS, over buy-only schedules, each taking the best of 1,000 random starting points on the constraint hyperplane. Their discretized cost coincides with (18), so at the settings  $X = 0.1$  and  $N = 100$  their reported optima and ours are minima of the same function. Two comparisons follow: against their published table, and against the same two algorithms rerun here to convergence.

The Adam descent reaches or passes every published value in 32 seconds on one NVIDIA H100 PCIe, 2.3 seconds for the twenty-one buy-only cells and 29.4 for the signed ones. Taken to the full budget of Table 1, at 696 seconds for the buy-only problem and 1,038 for the signed one, it lowers eleven of the signed cells below the published value and matches the other ten to within  $5 \times 10^{-5}$ , the precision to which Curato, Gatheral, and Lillo (2017) (CGL) report. At 29 seconds the signed schedules still stand as much as  $9.8 \times 10^{-4}$  above the costs they eventually attain, which is the size of the improvements themselves, so the four-decimal values quoted here need the longer run.

The CGL buy-only column stands further from the optimum, and the distance grows as  $\delta$  falls. At  $(\gamma, \delta) = (0.50, 0.50)$  the descent returns 0.02118 against a published 0.0278, a saving of 15.7% of the cost of trading at the constant rate  $X/T$ ; at  $(0.45, 0.55)$  it returns 0.01665 against 0.0212, a saving of 13.8%. Figure 4 plots the cost each method attains and Figure 5 the saving against the published value at every cell. GSS, rerun to convergence, reaches those same schedules: over the twenty-one cells it ends below the descent by at most  $5.3 \times 10^{-5}$ , the two

agree to eleven digits at the two cells with  $\delta = 1$ , and the descent ends lower at one cell. The distance between the CGL published buy-only column and the optimum is therefore a property of how far their runs were taken.

Note that optimal buy-only schedules are sparse, and the softmax of Example 3.3 sends every logit to a strictly positive rate. At  $(\gamma, \delta) = (0.50, 0.65)$ , the GSS schedule holds 81 of its 100 bins at exactly zero. Filling those 81 bins with a strictly positive value  $\varepsilon$  and rescaling, as in softmax, raises the exact cost by an amount proportional to  $\varepsilon^\delta$ .

Curato, Gatheral, and Lillo (2017) pass their objective to a library implementation of the method of Kolda, Lewis, and Torczon (2006), which receives function values alone and therefore, has to rebuild  $C$  at every candidate schedule. They report GSS to be the more expensive of their two methods. However, with an improved GSS method that exploits the structure of the objective, GSS becomes dramatically less expensive. A step of GSS moves rate between two bins, so the candidate differs from the current schedule in two coordinates, and (25) can compute its cost from two vectors that depend on the current schedule alone. Call the search that uses that form “improved GSS.” Improved GSS visits the same iterates as the published arrangement; ie. naive GSS and improved GSS return the same best cost, the same schedule, the same start index, and the same step and evaluation counts. The derivation and the implementation are in the accompanying source code.

On signed schedules the Adam descent attains the lower cost. Sequential quadratic programming (SQP), rerun at the thousand-start design, ends above the Adam descent at every one of the twenty-one cells, by up to  $1.6 \times 10^{-3}$ . The gap widens as  $\delta$  falls, along with the number of local minima the signed problem admits.

GSS, rebuilding  $C$  at every candidate, takes 11.4 hours on one core of an Apple M2 at  $(\gamma, \delta) = (0.45, 1.00)$  and 22.1 hours over the ten cells at  $\gamma = 0.45$ . Improved GSS returns the same schedules from 2,378 core-seconds at that cell, a factor of 17 better. Both methods evaluate one candidate at a time, so the remaining route to a shorter clock is to place the 1,000 starts on separate cores. Doing that with 26 cores, the whole grid takes 264 seconds and 1.8 core-hours for improved GSS, against 2,462 seconds and 17.6 core-hours for the quadratic program.

Those runs of classic algorithms hold 1,000 starts per cell. The Adam descent below holds 192,000 starts per cell, so the two clocks measure different amounts of search. The same descent, at the same 192,000 starts per cell and the same 8,000 steps, takes 11.0 hours on the Apple M2 when it is arranged as 480 sequential batches per cell, and 1,734 seconds on the H100 as one batch, a factor of 23 better. On the H100 GPU, one step of the batched descent is a single matrix product over 4,032,000 schedules, and the depth of the computation is simply the number of Adam steps.

### 3.7. Mapping the manipulation frontier

In this section, we take advantage of the fast computational methods of this paper to gain insight into the behavior of optimal cost for all values of  $(\delta, \gamma)$  in the unit square. A negative signed-acquisition value means that intermediate selling can turn a buy program into a source of revenue. A negative round-trip value gives a direct price-manipulation certificate. A numerical phase diagram shows how these two mechanisms appear on finite trading grids and how much resolution is needed to exhibit them.

The obstacle has been computational scale. At each exponent pair, one must solve a high-dimensional nonconvex optimization from enough initial schedules to find a useful basin. Curato,

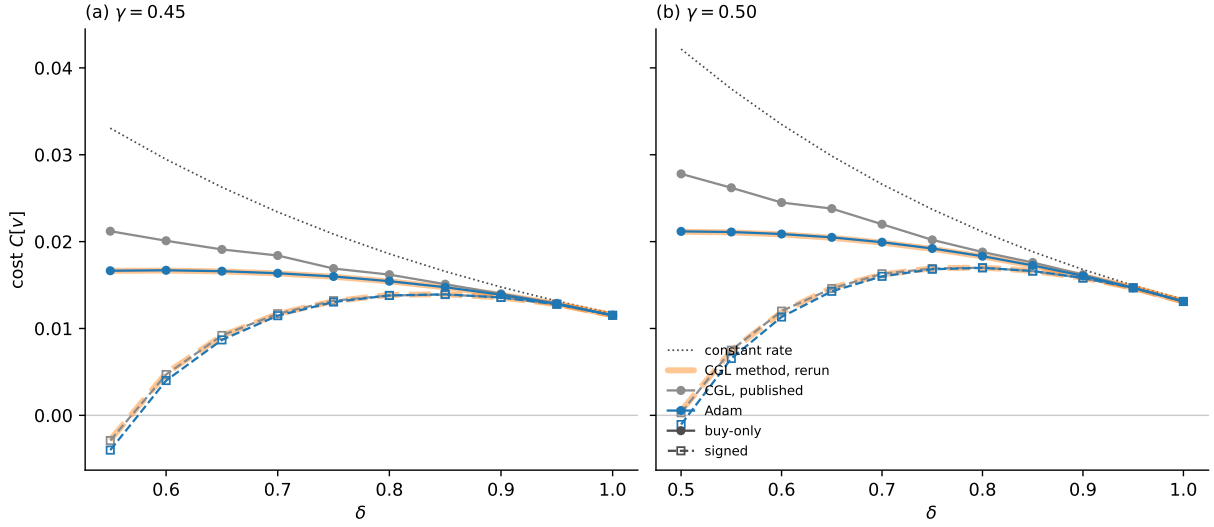


Figure 4: Best cost attained at  $X = 0.1$  and  $N = 100$ , with 1,000 starts for the classical methods and the budget of Table 1 for Adam. (a)  $\gamma = 0.45$ . (b)  $\gamma = 0.50$ . Filled markers and solid lines: buy-only schedules. Open markers and dashes: signed schedules. Dotted: the cost of trading at the constant rate  $X/T$ .

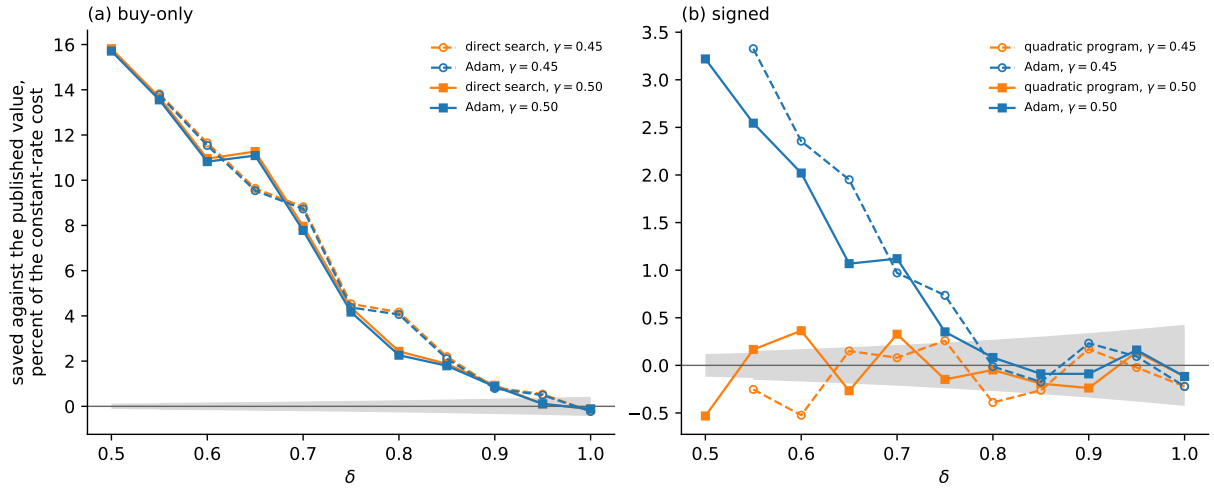


Figure 5: Cost saved against Curato, Gatheral, and Lillo (2017), Table 2, as a percentage of the cost of trading at the constant rate  $X/T$ .  $X = 0.1$ ,  $N = 100$ , 1,000 starts for the classical methods and the budget of Table 1 for Adam. (a) Buy-only schedules, against their direct search column. (b) Signed schedules, against their quadratic program column. Open markers and dashes:  $\gamma = 0.45$ . Filled markers and solid lines:  $\gamma = 0.50$ . Shaded: the four-decimal resolution of the published table.

Gatheral, and Lillo (2017) established the pointwise benchmark with one thousand starts per cell. Extending that design to a dense exponent plane calls for millions of separate local optimizations. The batched descent of Section 3.2 allows us to perform those millions of local optimizations efficiently.

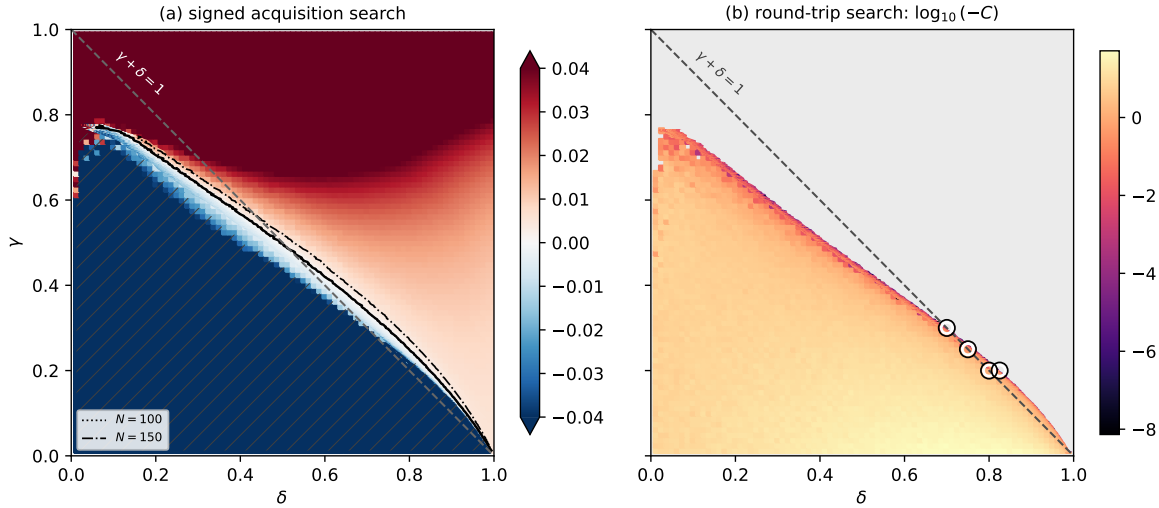


Figure 6: The exponent plane at  $X = 0.1$ . Background:  $N = 100$  and spacing 0.0125. Boundary bands:  $\delta$  spacing 0.0025 and up to twenty-four restarts. Dashed line:  $\gamma + \delta = 1$ . (a) Signed acquisition search; solid:  $N = 100$  zero contour on the fine mesh; broken: zero contours at  $N = 100, 150, 300, 600$  on the common coarse mesh; hatched: cells where panel (b) certifies a round trip, so that the acquisition infimum is  $-\infty$  by Proposition 3.4. (b) Magnitude of certified negative-cost round trips; grey: no negative-cost round trip was found.

**Proposition 3.4.** *Assume the power-law model (5) with  $\delta < 1$ , on a grid of  $N$  bins with target  $X \in \mathbb{R}$ . If some piecewise-constant round trip  $u$  has  $C[u] < 0$ , then*

$$\inf\{C[v] : \Delta \sum_n v_n = X\} = -\infty.$$

*Proof.* Fix a feasible  $v^0$  and put  $v^s = v^0 + su$  for  $s > 0$ , feasible because  $u$  has zero net quantity. Since  $f$  is positively homogeneous of degree  $\delta$ , each coordinate obeys  $f(v_m^s) = s^\delta f(u_m) + O(1)$  as  $s \rightarrow \infty$ . Substituting into (25) and collecting powers of  $s$  gives  $C[v^s] = s^{1+\delta}C[u] + O(s)$ , and  $1 + \delta > 1$  with  $C[u] < 0$  sends the leading term to  $-\infty$ .  $\square$

Wherever panel (b) is colored, then, the minimization behind panel (a) has infimum  $-\infty$ . A search there reports how far it descended within its budget. Refining the trading grid supplies more directions in which to descend, and the reported values fall without settling: at  $(\gamma, \delta) = (0.30, 0.70)$  they run  $-0.063, -0.929, -8.94, -14.79$  across  $N = 100, 150, 300, 600$ . Panel (a) therefore locates the region where the acquisition problem is unbounded. Panel (b) supplies the certificates, every colored cell holding an exact negative-cost round trip, so that region is an inner bound on the manipulable set.

Figure 7 turns the phase diagram into one-dimensional cross-sections. In panel (a), the  $\delta = 0.50$  signed-acquisition curve crosses zero well above  $\gamma = 1 - \delta$ . In panel (b), the cost bends nonmonotonically with  $\delta$ , reproducing the shape reported by Curato, Gatheral, and Lillo (2017).

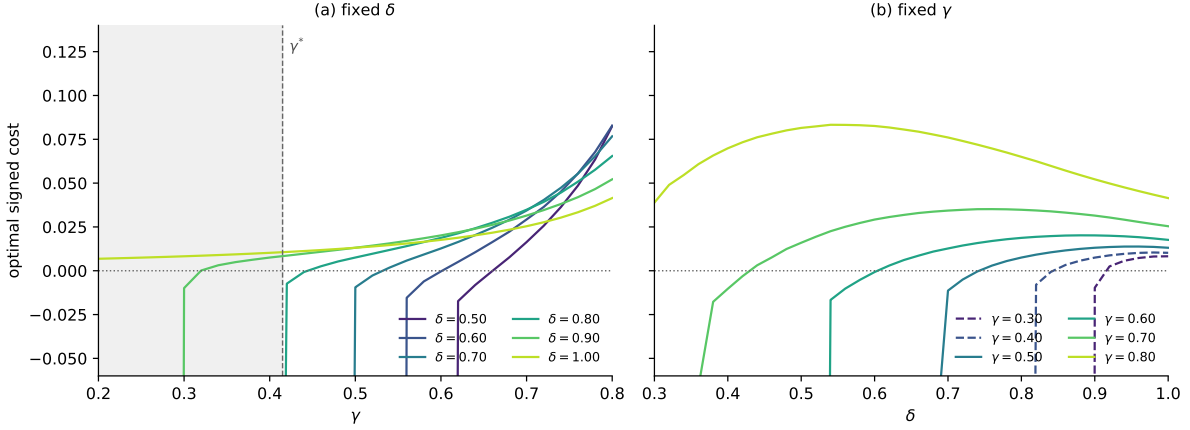


Figure 7: Optimal signed cost slices at  $N = 3,200$  and  $X = 0.1$ , with 6,000 steps per restart and spacing 0.02 in the running exponent. (a) Cost against  $\gamma$  at fixed  $\delta$ , twelve restarts; shaded:  $\gamma < \gamma^*$ . (b) Cost against  $\delta$  at fixed  $\gamma$ , at least twenty-four restarts; dashed:  $\gamma < \gamma^*$ . Dotted line: zero.

Negative signed values are continuous-time certificates for transaction-triggered manipulation, while Figure 6(b) supplies the round-trip certificates. At  $\delta = 1$ , the exact convex-program value remains positive throughout.

Batched automatic differentiation makes it possible to compute the phase diagram in a reasonable amount of time on neural network-optimized hardware, and Lemma 2.6 turns the schedules it finds into certificates for the continuous-time model. Within the resolution of the computation this matches the pattern predicted by Gatheral (2010): the diagonal  $\gamma + \delta = 1$  governs most of the certified region, and the certificates that lie strictly above it fall in the regime  $\gamma < \gamma^*$ , at  $\gamma$  between 0.0375 and 0.2375. What the map delivers is one side of the question. Every colored cell is a manipulable pair, established by exhibiting a schedule; a cell left grey means only that this search found nothing, and Proposition 3.4 shows that once a cell is colored the acquisition problem beside it has no finite optimum at all.

### 3.8. Slippage

Let the impacted mid-price  $S(t)$  be the propagator process (1), including transient impact. We assume the existence of *slippage*, which means that a market order is filled at an execution price  $\tilde{S}(t)$  with an additional instantaneous concession beyond the mid-price  $S(t)$ . The classic model of slippage (Almgren and Chriss, 2001) includes a linear concession proportional to the rate and a spread-like concession paid in the direction of the trade:

$$\tilde{S}(t) = S(t) + \eta_1 v(t) + \eta_2 \operatorname{sgn}(v(t)), \quad \eta_1, \eta_2 \geq 0. \quad (30)$$

The two terms are the two components of the temporary-impact function of Almgren and Chriss (2001). Trading against the execution price adds an instantaneous term to the cost:

$$\tilde{\mathcal{C}}[v] = \mathbb{E} \int_0^1 v(t) (\tilde{S}(t) - S(0)) dt = \mathcal{C}[v] + \eta_1 \int_0^1 v(t)^2 dt + \eta_2 \int_0^1 |v(t)| dt. \quad (31)$$

Following the convention of Definition 2.7, the roman counterpart is the value of (31) on the grid. For  $v$  piecewise constant on the  $N$  bins the two slippage integrals are exact sums, and Lemma 2.6 gives the propagator term, so

$$\tilde{C}_\eta[v] = C[v] + \eta_1 \Delta \sum_{n=1}^N v_n^2 + \eta_2 \Delta \sum_{n=1}^N |v_n|, \quad (32)$$

and  $\tilde{C}_\eta[v] = \tilde{C}[v]$  for every such  $v$ . The subscript is written only where the dependence on  $\eta = (\eta_1, \eta_2)$  is being varied; elsewhere  $\eta$  is held fixed and the cost is written  $\tilde{C}[v]$ .

Write

$$h(x_n) = \eta_1 v_n + \eta_2 \operatorname{sgn}(v_n) \quad (33)$$

for the slippage in interval  $n$ , where as before,  $x_n = (v_n, c_n)$  denotes the full token. In machine learning terminology,  $h$  is a position-wise head attached to the current token and the network structure sends the output of  $h$  only to the execution-price readout and the loss. The impact state and the keys and values for later tokens follow a separate causal path not affected by  $h$ , as illustrated by Figure 8.

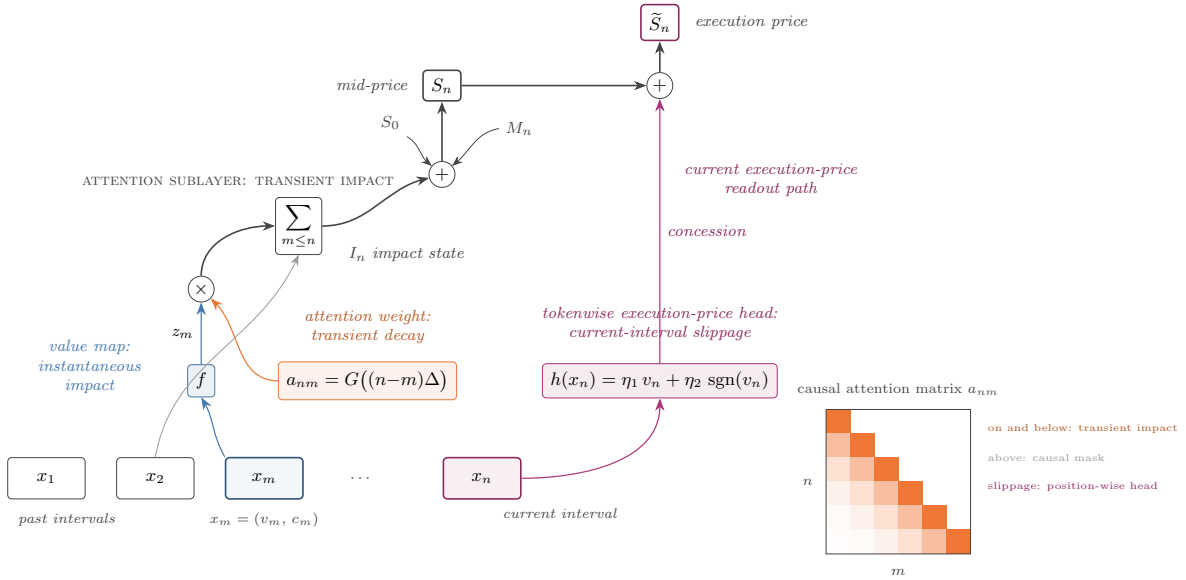


Figure 8: Transient impact and slippage in the execution architecture. Blue: instantaneous-impact value map. Orange: transient-decay weights. Purple: position-wise concession in the current execution price  $\tilde{S}_n$ .

Suppose we have found a model which allows price manipulation in the form of a negative-cost round trip; this is Gatheral’s notion of dynamic arbitrage. In any such model, one can then achieve arbitrarily negative cost by scaling the profitable round trip. It is natural to ask how this changes when slippage terms of the form (30) are added to the model and, in particular, whether adding slippage terms can make the cost-minimization problem well-posed when it wasn’t before. This is answered in Proposition 3.5; it turns out that  $\eta_1 > 0$  is necessary here.

**Proposition 3.5.** *Work in the power-law model (5) with  $\delta < 1$ , on a grid of  $N$  bins with target  $X \in \mathbb{R}$ . Suppose some piecewise-constant round trip  $u$  has  $C[u] < 0$ , and consider the slippage cost (32) on the acquisition set  $\{v \in \mathbb{R}^N : \Delta \sum_n v_n = X\}$ .*

(i) *If  $\eta_1 = 0$ , then  $\inf_v \tilde{C}[v] = -\infty$  for every  $\eta_2 \geq 0$ .*

(ii) *If  $\eta_1 > 0$ , then  $\inf_v \tilde{C}[v]$  is finite and is attained.*

*Proof.* For (i), take the path  $v^s = v^0 + su$  of Proposition 3.4, along which  $C[v^s] = s^{1+\delta}C[u] + O(s)$ . The spread term satisfies

$$\eta_2 \Delta \sum_n |v_n^s| = O(s),$$

so  $\tilde{C}[v^s] = s^{1+\delta}C[u] + O(s)$  as well. Because  $C[u] < 0$  and  $1 + \delta > 1$ , the leading term dominates and  $\tilde{C}[v^s] \rightarrow -\infty$ .

For (ii), write  $\|\cdot\|$  for the Euclidean norm on  $\mathbb{R}^N$  and  $\|\mathbf{W}\|$  for the operator norm of the matrix in (25). Hölder's inequality with exponents  $1/\delta$  and  $1/(1-\delta)$  gives  $\sum_m |v_m|^{2\delta} \leq N^{1-\delta} \|v\|^{2\delta}$ , hence  $\|f(v)\| \leq \lambda N^{(1-\delta)/2} \|v\|^\delta$ , and Cauchy–Schwarz gives

$$|C[v]| = |v^\top \mathbf{W} f(v)| \leq K \|v\|^{1+\delta}, \quad (34)$$

where the constant is  $K := \lambda N^{(1-\delta)/2} \|\mathbf{W}\|$ . Discarding the nonnegative spread term leaves  $\tilde{C}[v] \geq \eta_1 \Delta \|v\|^2 - K \|v\|^{1+\delta}$ . The exponent  $1 + \delta$  is smaller than 2, so the right side is bounded below on  $[0, \infty)$  and tends to  $+\infty$  with  $\|v\|$ . Thus  $\tilde{C}$  is coercive and bounded below on the feasible set, a closed affine subspace, and a continuous coercive function on a closed set attains its infimum.  $\square$

The infimum in (ii) can be negative. The bound in the proof is itself negative at small  $\|v\|$ , since  $1 + \delta < 2$  makes  $K \|v\|^{1+\delta}$  exceed  $\eta_1 \Delta \|v\|^2$  there. What  $\eta_1 > 0$  buys is a finite optimum, and whether that optimum is profitable is the question the rest of this section answers.

Numerically, we verified the existence of a negative cost round trip at  $(\gamma, \delta) = (0.30, 0.70)$ . By Proposition 3.5(i), the edge  $\eta_1 = 0$  therefore has infimum  $-\infty$  at every  $\eta_2 \geq 0$ , and it is excluded from Figure 9. At every  $\eta_1 > 0$  the infimum is finite and attained, and on 948 of the 6,480 cells of Figure 9 with  $\eta_1 > 0$  it is strictly negative.

The acquisition set  $F = \{v \in \mathbb{R}^N : \Delta \sum_n v_n = X\}$  does not depend on  $\eta$ . Put

$$Q[v] = \Delta \sum_n v_n^2, \quad S[v] = \Delta \sum_n |v_n|,$$

so that (32) reads  $\tilde{C}_\eta[v] = C[v] + \eta_1 Q[v] + \eta_2 S[v]$ . For fixed  $v$  this is an affine function of  $\eta$  with nonnegative coefficients.

A search at one value of  $\eta = (\eta_1, \eta_2)$  returns one schedule, and that schedule lies in  $F$ . It is therefore admissible at every other value of  $\eta$ , and its cost there follows from the three numbers  $C[v]$ ,  $Q[v]$ ,  $S[v]$  without any further optimization. Let  $\mathcal{L}$  be the finite set of schedules returned over all cells and define

$$\hat{C}(\eta) = \min_{v \in \mathcal{L}} (C[v] + \eta_1 Q[v] + \eta_2 S[v]). \quad (35)$$

Three properties follow. First,  $\hat{C}(\eta) \geq \inf_{v \in F} \tilde{C}_\eta[v]$ , because  $\mathcal{L} \subseteq F$ . Second,  $\hat{C}(\eta)$  is at most the value found by the search at  $\eta$  itself, because that schedule is one element of  $\mathcal{L}$ . Third,  $\hat{C}$  is a minimum of finitely many affine functions with nonnegative gradients, so it is nondecreasing

in  $\eta_1$  and in  $\eta_2$ . A minimum of affine functions is also concave, so the set  $\{\eta : \widehat{C}(\eta) \geq 0\}$ , where the pooled library holds nothing profitable, is convex. Also, it contains  $\eta'$  whenever it contains some  $\eta \leq \eta'$ . Its lower boundary is the zero contour, and convexity places the segment joining any two points of that contour above it.

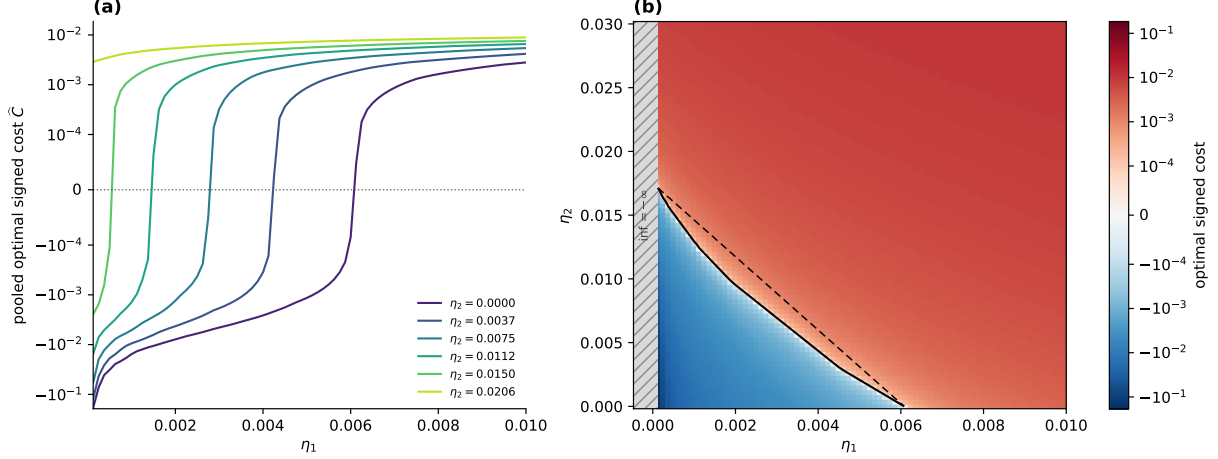


Figure 9: Pooled optimal signed cost  $\widehat{C}$  of (35) at  $\gamma = 0.30$ ,  $\delta = 0.70$ ,  $X = 0.1$ ,  $N = 100$ , over an  $81 \times 81$  grid in  $(\eta_1, \eta_2)$  with  $\eta_1 = 0$  excluded. (a)  $\widehat{C}$  against  $\eta_1$  at six fixed  $\eta_2$ , on a symmetric logarithmic vertical scale with linear threshold  $10^{-4}$ ; dotted line: zero. (b)  $\widehat{C}$  over the plane, on the same symmetric logarithmic scale in colour, blue negative and red positive. Solid: the zero contour. Dashed: the chord from  $(\eta_1, \eta_2) = (6.08 \times 10^{-3}, 0)$  to  $(1.25 \times 10^{-4}, 1.71 \times 10^{-2})$ , joining the two intercepts of that contour. Hatched strip at the left edge of (b):  $\eta_1 = 0$ , where the infimum is  $-\infty$  by Proposition 3.5(i).

Figure 9 locates that boundary at  $(\gamma, \delta) = (0.30, 0.70)$  and shows what the schedules on either side of it do. The zero contour meets  $\eta_2 = 0$  at  $\eta_1 = 6.08 \times 10^{-3}$  and reaches  $\eta_2 = 1.71 \times 10^{-2}$  at the smallest plotted  $\eta_1$ . Those two intercepts are the cost of removing the manipulation with one concession alone. Convexity puts the chord joining them inside  $\{\widehat{C} \geq 0\}$ , so a mixture of the two concessions suffices at every intermediate point, and the contour runs inside the chord by up to  $7.0 \times 10^{-4}$  in  $\eta_1$ .

Panel (a) cuts the same surface along  $\eta_1$  at six fixed values of  $\eta_2$ , in the manner of Figure 7. Each slice rises with  $\eta_1$  and crosses zero once, at  $\eta_1 = 6.08, 4.23, 2.78, 1.47$  and  $0.53$ , all  $\times 10^{-3}$ , as  $\eta_2$  runs from 0 to  $1.50 \times 10^{-2}$ ; those crossings are the frontier read off the horizontal axis. At  $\eta_2 = 2.06 \times 10^{-2}$ , above the intercept  $\eta_2^*$ , the slice stays nonnegative for every  $\eta_1$ . The vertical scale is symmetric logarithmic with the same linear threshold  $10^{-4}$  as the colours of panel (b), so the depth of the negative region is legible: at  $\eta_2 = 0$  the cost reaches  $-0.185$ , two orders of magnitude below the positive values it takes once  $\eta_1$  passes the crossing.

#### 4. Training on metaorder data

The natural training set for the model is a collection of metaorders together with their fills and the surrounding prices, and it maps onto the architecture directly. A training sample corresponds to one metaorder: an execution window discretized into bins of width  $\Delta$ , the signed

filled quantity  $q_n$  in bin  $n$  defining the realized trading rate  $v_n = q_n/\Delta$ , the token  $x_n$  collecting  $v_n$  together with public covariates such as volatility, spread, depth, and volume, and the mid-price path  $S_0, \dots, S_{N+P}$  observed both during execution and for  $P$  post-completion bins in which  $v = 0$ . The post-completion bins are included because there the head predicts pure relaxation of the accumulated impact state, so they pin down the decay side of the kernel separately from the instantaneous side. Metaorders of different lengths are padded and masked, and the dataset becomes a tensor of token sequences.

Fitting in dimensionless units removes effects due to price levels and volatility. Deflate by the arrival price  $S_0^{(i)}$ , held fixed across the window of metaorder  $i$ , so that the model stays additive and the impact state remains the sum (20) rescaled. Write

$$r_n^{(i)} := \frac{S_n^{(i)} - S_{n-1}^{(i)}}{S_0^{(i)}}, \quad (36)$$

for the return observed over bin  $n$  and

$$\Delta \iota_n^{(i)}(\theta) := \frac{I_n^{(i)}(\theta) - I_{n-1}^{(i)}(\theta)}{S_0^{(i)}} \quad (37)$$

for the return the model attributes to impact.

The model (10) asserts  $r_n = \Delta \iota_n(\theta) + \Delta \mu_n$ , where  $\mu_n = M_n/S_0$  is a conditionally mean-zero martingale, so with  $\varsigma_n^{(i)}$  the standard deviation of  $r_n^{(i)}$ ,

$$\mathcal{L}_{\text{fit}}(\theta) = \sum_{i \in \text{batch}} \sum_{n=1}^{N_i+P} \left( \frac{r_n^{(i)} - \Delta \iota_n^{(i)}(\theta)}{\varsigma_n^{(i)}} \right)^2, \quad (38)$$

is a quasi-maximum-likelihood objective, minibatched over metaorders. Every term is a squared standard score, so a batch may mix assets, price levels and dates.

The Adam descent algorithm (Kingma and Ba, 2015) minimizes the training loss in PyTorch (Paszke et al., 2019) through the same computational graph that Section 3 differentiates to solve for schedules, so a single implementation of the forward pass serves both estimation and execution.

By Definitions 2.1 and 2.2 the impact function is odd, vanishes at the origin and is non-decreasing, while the propagator is positive and nonincreasing. Restrictions of this kind are often imposed by projecting after each step or by adding a penalty, and both interfere with the descent. Choosing coordinates in which every point of the parameter space is admissible is preferable because the optimizer then runs unconstrained, and every iterate, including any early stopping point, is a legitimate model.

For the propagator the mixture (12) suggests coordinates that map onto admissible propagators. Let  $a, b \in \mathbb{R}^K$  be unconstrained, and set  $\omega_k = \text{softplus}(a_k)$  and  $\rho_k = e^{b_k}$ ; the kernel is then positive, nonincreasing and completely monotone for every  $(a, b)$ . Increasing  $K$  gives finite-mixture approximations to power-law decay on bounded ranges. For the impact function, build it from a nonnegative derivative in  $|v|$  and extend it oddly. A sum of increasing logistic increments with nonnegative coefficients is one such family,

$$f_\theta(v) = \text{sgn}(v) \sum_{j=1}^J \text{softplus}(c_j) \left[ \frac{1}{1 + \exp(-\text{softplus}(s_j)|v| - d_j)} - \frac{1}{1 + \exp(-d_j)} \right], \quad (39)$$

with  $c, s, d \in \mathbb{R}^J$  unconstrained. Every member is odd, vanishes at the origin and is nondecreasing in  $|v|$ , and the family saturates at large rates. Imposing concavity as well amounts to

making the derivative decreasing, a further shape constraint of the same kind.

The value map of Section 5.1,

$$f(x_m) = Y \sigma_m \operatorname{sgn}(v_m) \left| \frac{v_m}{V_m} \right|^\delta,$$

depends on the prevailing volatility  $\sigma_m$  and the market volume rate  $V_m$ , which would be among the covariates  $c_m$  and hence part of the token  $x_m$ . This leaves only  $Y$  and  $\delta$  to estimate. Taking  $Y = e^{\theta_Y}$  and  $\delta = (1 + e^{-\theta_\delta})^{-1}$  carry an unconstrained  $(\theta_Y, \theta_\delta) \in \mathbb{R}^2$  onto  $Y > 0$  and  $\delta \in (0, 1)$ , and every member of the family is odd, vanishes at the origin, and is nondecreasing and concave in  $|v|$  with  $\delta < 1$ . These parameterizations feed directly into the differentiable forward pass.

```
G = lambda t, a, b: (softplus(a) * exp(-exp(b) * t[... , None])).sum(-1)
f = lambda v, c, s, d: sign(v) * (softplus(c) *
                                (sigmoid(softplus(s)*abs(v)[... , None] + d)
                                 - sigmoid(d))).sum(-1)

parameters = [a, b, c, s, d] # each has requires_grad=True
opt = Adam(parameters, lr)

for _ in range(steps):
    opt.zero_grad(set_to_none=True)
    weights = G(lag, a, b)
    values = f(v_observed, c, s, d)
    impact = causal_head(weights, values, mask, initial=0)
    predicted_return = diff(impact, dim=-1) / S0
    residual = (observed_return - predicted_return) / sigma
    loss = (mask * residual**2).sum()
    loss.backward()
    opt.step()
```

The call to `loss.backward()` traverses the causal head, the kernel weights and the value map. It calculates and stores derivatives with respect to  $a, b, c, s, d$  in one reverse sweep. As the number of assets and trading venues continues to increase, as does the quality of machine-learning hardware, the sheer size of typical training sets is likely to explode and we expect the fitting technique of this section to be increasingly helpful.

## 5. Discussion and open problems

The correspondence developed here locates the classic propagator model within the modern sequence-model taxonomy, as a one-layer, single-head, causal, position-only unnormalized kernel-attention network, with a concave value map. Hence the impact model and the execution problem become one differentiable object: the same forward pass through the network prices a schedule, fits the kernel to data, and supplies the gradients for both.

This correspondence may continue to pay dividends down the road, but an immediate benefit is that it clarifies how to adapt the classical propagator model to hardware and software optimized for machine-learning workflows. In particular, we define and use the appropriate version of batched Adam descent which allows us to perform large-scale grid searches over the full parameter space that would have been computationally infeasible just a few years ago. For example, the large “cost landscape” maps such as Figures 6 and 9 partition cleanly across several

devices, which cloud providers now supply as eight-GPU nodes, so the efficacy of our methods must continue to grow as hardware improves.

Each subsection below generalizes the classical propagator model by replacing one fixed ingredient by a learnable one: the value map by a network on the whole token, the positional decay by a state-dependent weight, the single asset by a matrix kernel over assets, and the slippage coefficients by trainable parameters. In our view, the most important set of open problems is to investigate these generalizations on real metaorder data, using the computational methods of Section 4.

### 5.1. *Covariates in the value map: forecasting impact*

The classical estimation literature already uses hand-designed covariate-dependent value maps without calling them that explicitly. Writing  $\sigma_m$  for the prevailing volatility and  $V_m$  for the market volume, both coordinates of the covariate block  $c_m$ , the square-root/power-law impact formulas of Almgren et al. (2005) and Lillo, Farmer, and Mantegna (2003) take the form

$$f(x_m) = Y \sigma_m \operatorname{sgn}(v_m) \left| \frac{v_m}{V_m} \right|^\delta,$$

so that impact is measured in volatility units and driven by the participation rate. Both sides are then dimensionless and  $Y$  is a pure number, in contrast with the impact coefficient  $\lambda$  of (3), which carries the units of currency per share squared; at  $\delta = 1$  the two are related by  $\lambda = Y \sigma_m / V_m$ . In machine learning terminology, this is a specific two-covariate value map. The transformer generalization replaces it by a multilayer perceptron on the full token, which can learn the classical scalings and their corrections: dependence on spread, book depth and imbalance, time of day, recent signed flow, and announced-event dummies. Because the value map acts position-wise, this extension alone, with  $A_\theta(x, y, \tau) = G_\theta(\tau)$ , yields a strictly richer forecasting model of impact while remaining a propagator model in structure.

### 5.2. *State-dependent impact decay*

The classical price response at  $t$  to a trade at  $s$  depends on  $t - s$  alone. Empirically, the relaxation of impact appears to depend on subsequent order flow, realized volatility, and liquidity conditions (Bouchaud et al., 2004; Bouchaud et al., 2018). Impact appears to relax faster when it occurs in the context of opposing flow, and more slowly in quiet markets. The factorization  $A_\theta(x_t, x_s, \tau) = \langle \Phi_\theta(x), \Psi_\theta(y) \rangle G_\theta(\tau)$  of (21) is flexible enough to express this. The query features  $\Phi_\theta(x_n)$  read the current liquidity state, the key features  $\Psi_\theta(x_m)$  read the state in which the impact was created, and their inner product modulates the positional decay  $G_\theta(\tau)$ .

### 5.3. *Cross-impact: attention over assets*

For  $N$  assets with trading-rate vector  $v_s \in \mathbb{R}^N$ , the linear multi-asset propagator (Benzaquen et al., 2017) takes

$$S_t^i - S_0^i = \sum_j \int_0^t G_{ij}(t - s) v^j(s) ds + \text{noise},$$

with a matrix kernel  $G(\tau) \in \mathbb{R}^{N \times N}$ . In our terminology, this is attention over the pair (asset, time). More specifically we say token  $(j, s)$  is *attended to* by  $(i, t)$  with weight  $G_{ij}(t - s)$ .

Taking the feature dimension  $p$  of (21) small expresses cross-impact through a correspondingly small number of liquidity factors, matching the empirical finding that cross-impact matrices are approximately low-rank with a dominant market mode (Benzaquen et al., 2017).

#### 5.4. *Learnable slippage parameters*

When training on metaorder data as in Section 4, one can of course take a classic slippage model (33) and include  $\eta_1, \eta_2$  among the trainable parameters. The computational framework of transformer models would also naturally allow more general parameterized functional forms in place of (33), making the slippage function  $h$  a learnable function and potentially dependent upon the full token  $x_n$  rather than only on the trading rate.

#### *Code availability*

The scripts that produce every figure and number in this paper are available at:

<https://github.com/gordonritter/transformer-propagator>.

The benchmark searches write their results to JSON files that are included in the repo, and the JSON files are regenerated only if missing. The procedure for running the batched Adam descents on a datacenter-hosted NVIDIA GPU is described in LAMBDA.md.

#### *Declarations of Interest*

The authors report no conflicts of interest. The authors alone are responsible for the content and writing of the paper.

#### *Acknowledgements*

Claude and ChatGPT were used as coding assistants to verify and optimize the accompanying python code, as grammar checkers, and to prepare Figures 1, 2 and 8. All original ideas are the author’s own.

#### **References**

- Alfonsi, Aurélien, Alexander Schied, and Alla Slynko (2012). “Order book resilience, price manipulation, and the positive portfolio problem”. In: *SIAM Journal on Financial Mathematics* 3.1, pp. 511–533.
- Almgren, Robert and Neil Chriss (2001). “Optimal execution of portfolio transactions”. In: *Journal of Risk* 3.2, pp. 5–39.
- Almgren, Robert et al. (2005). “Direct estimation of equity market impact”. In: *Risk* 18.7, pp. 58–62.
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2015). “Neural machine translation by jointly learning to align and translate”. In: *International Conference on Learning Representations*.

- Benzaquen, Michael et al. (2017). “Dissecting cross-impact on stock markets: an empirical analysis”. In: *Journal of Statistical Mechanics: Theory and Experiment* 2017.2, p. 023406.
- Bouchaud, Jean-Philippe et al. (2004). “Fluctuations and response in financial markets: the subtle nature of “random” price changes”. In: *Quantitative Finance* 4.2, pp. 176–190.
- Bouchaud, Jean-Philippe et al. (2018). *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge University Press.
- Curato, Gianbiagio, Jim Gatheral, and Fabrizio Lillo (2017). “Optimal execution with non-linear transient market impact”. In: *Quantitative Finance* 17.1, pp. 41–54.
- Gatheral, Jim (2010). “No-dynamic-arbitrage and market impact”. In: *Quantitative Finance* 10.7, pp. 749–759.
- Griewank, Andreas and Andrea Walther (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2nd. SIAM.
- Huberman, Gur and Werner Stanzl (2004). “Price manipulation and quasi-arbitrage”. In: *Econometrica* 72.4, pp. 1247–1275.
- Katharopoulos, Angelos et al. (2020). “Transformers are RNNs: fast autoregressive transformers with linear attention”. In: *International Conference on Machine Learning*.
- Kingma, Diederik P. and Jimmy Ba (2015). “Adam: a method for stochastic optimization”. In: *International Conference on Learning Representations*.
- Kolda, Tamara G., Robert Michael Lewis, and Virginia Torczon (2003). “Optimization by Direct Search: New Perspectives on Some Classical and Modern Methods”. In: *SIAM Review* 45.3, pp. 385–482.
- (2006). “Stationarity Results for Generating Set Search for Linearly Constrained Optimization”. In: *SIAM Journal on Optimization* 17.4, pp. 943–968.
- Kyle, Albert S (1985). “Continuous auctions and insider trading”. In: *Econometrica: Journal of the Econometric Society*, pp. 1315–1335.
- Leal, Laura, Mathieu Laurière, and Charles-Albert Lehalle (2022). “Learning a functional control for high-frequency finance”. In: *Quantitative Finance* 22.11.
- Lillo, Fabrizio, J. Doyne Farmer, and Rosario N. Mantegna (2003). “Master curve for price-impact function”. In: *Nature* 421, pp. 129–130.
- Neuman, Eyal, Wolfgang Stockinger, and Yufei Zhang (2023). *An offline learning approach to propagator models*. arXiv:2309.02994.
- Paszke, Adam et al. (2019). “PyTorch: an imperative style, high-performance deep learning library”. In: *Advances in Neural Information Processing Systems*.
- Schilling, René L., Renming Song, and Zoran Vondraček (2012). *Bernstein Functions: Theory and Applications*. 2nd. De Gruyter.
- Tsai, Yao-Hung Hubert et al. (2019). “Transformer dissection: a unified understanding of transformer’s attention via the lens of kernel”. In: *Conference on Empirical Methods in Natural Language Processing*.
- Tsaknaki, Ioanna-Yvonne, Andrea Macrì, and Fabrizio Lillo (2026). *Can reinforcement learning efficiently discover price manipulation?* arXiv:2607.06121.
- Vaswani, Ashish et al. (2017). “Attention is all you need”. In: *Advances in Neural Information Processing Systems*.
- Webster, Kevin T. (2023). *Handbook of Price Impact Modeling*. Chapman and Hall/CRC.
- Widder, David Vernon (1941). *The Laplace Transform*. Princeton University Press.